Power Query Editor – Additional Features

Power Query Editor in Power BI is a powerful data transformation engine that enables users to shape, clean, and prepare their datasets before loading them into the Power BI data model. While many foundational features—such as promoting headers, changing data types, splitting columns, merging, and appending—are widely known, Power Query also provides a set of **advanced and intermediate column-level tools** that offer deeper customization. These tools help analysts automate logic, create new fields, clean complex data, or build dynamic parameters that can be reused across multiple transformations.

This document explores the following features in detail:

1. **Conditional Column**
2. **Index Column**
3. **Duplicate Column**
4. **Parse Options (XML/JSON)**
5. **Invoke Custom Function**
6. **Column From Examples**
7. **Manage Parameters**

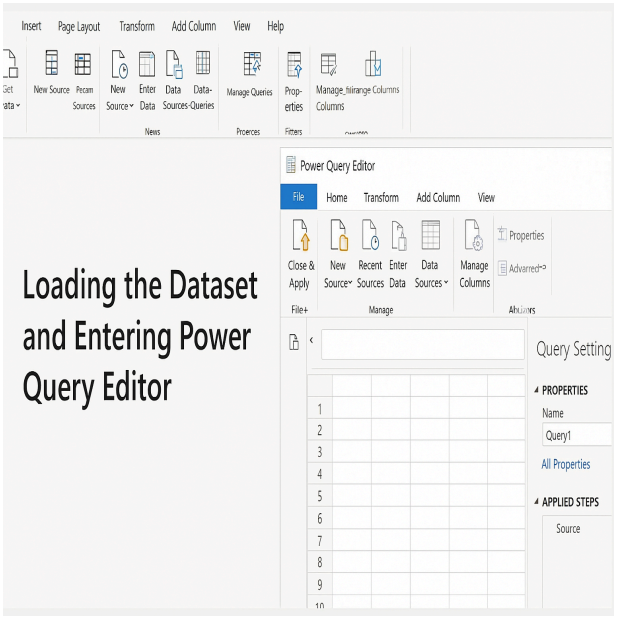
YATICORP
EMPOWERING EDUCATION

The explanation is based on importing a CSV file named **SCM_Orders.csv** into Power Query and performing transformations using these advanced features.

---

1. Loading the Dataset and Entering Power Query Editor

After creating a blank Power BI report, the process typically begins by selecting **Get Data → Text/CSV**, followed by choosing the file *SCM_Orders.csv*. Power BI provides a data preview; from here, clicking **Transform Data** opens the dataset in the **Power Query Editor**, where all data shaping operations take place before loading into Power BI.



Inside Power Query, the interface includes several tabs such as:

- **Home**
- **Transform**
- **Add Column**
- **View**

The features discussed in this document primarily reside in the **Add Column** and **Home** tabs.

---

2. Conditional Column

A **Conditional Column** lets users create a **new calculated column** whose value depends on logical conditions applied to an existing column—similar to IF, ELSEIF, and ELSE constructs in Excel or programming languages.

Use Case Example – Reclassifying Category Names

YATICORP
EMPOWERING EDUCATION

Given the *Category* column in the SCM_Orders dataset, suppose the company decides to standardize category names:

- **Furniture → Furn**
- **Office Supplies → OS**
- **Technology → Tech**

To implement this logic:

Steps:

1. Go to **Add Column → Conditional Column**.
2. Name the new column (e.g., *New Category Name*).
3. Select the column to evaluate (*Category*).
4. Build the conditions:
    - If Category equals *Furniture*, output **Furn**

- Else if Category equals *Office Supplies*, output **OS**
- Else (default), output **Tech**

The final condition does not need an explicit check for *Technology* if only three categories exist.

Output:

A new column is created showing the converted category names for each row.

Key Advantages:

- Provides a **user-friendly interface** to build logical tests.
- Avoids manual coding in M language.
- Works for both numeric and text-based conditional logic.
- Conditions are stored under the *Applied Steps*, allowing easy editing.

YATICORP
EMPOWERING EDUCATION

3. Index Column

An **Index Column** generates automatic sequential numbering for each row. This is often useful when:

- You need a unique row identifier
- Sorting must be preserved
- Serial numbers need to be displayed in the report
- Data needs to be referenced in custom computations

Steps to Add Index Column:

1. Go to **Add Column → Index Column**.
2. Choose from:
    - **From 0**
    - **From 1**

- **Custom** starting value and increment

In the example, **From 1** is selected, generating values 1, 2, 3, …

Post-Creation Adjustments:

- Rename the column (e.g., *Serial Number*).
- Change data type (often text for categorical identification).
- Right-click → **Move** → **To Beginning** to position it as the first column.

Benefits:

- Helps track rows after sorting or filtering.
- Used in advanced transformations that require references to previous rows.
- Enhances clarity in tabular reports.

YATICORP
EMPOWERING EDUCATION

4. Duplicate Column

The **Duplicate Column** feature quickly creates a copy of an existing column.

Why Duplicate a Column?

- To apply transformations on a copy while retaining the original.
- To build calculations based on differently formatted versions of the same data.
- To prepare data for operations like splitting, parsing, or tokenization.

Steps:

1. Select the column.
2. Go to **Add Column** → **Duplicate Column**, or right-click → **Duplicate**.

This creates a new column with the same values, which can then be transformed independently.

---

5. Parse Options – XML and JSON

The **Parse** category in Power Query allows users to extract structured elements from:

- XML data
- JSON data

Although not used in this specific demonstration (CSV file), this feature is essential for **advanced integration** with:

- APIs
- Web services
- Application logs
- Nested data formats

YATICORP
EMPOWERING EDUCATION

Examples of What Parse Can Do:

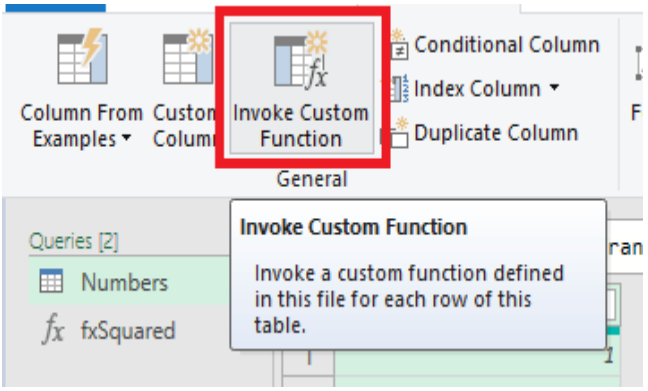
- Convert a JSON string into expandable records and lists.
- Convert XML content into structured tables.
- Automatically detect hierarchical relationships.

This tool is intended for analysts working with webpages, API responses, or unstructured data sources.

---

6. Invoke Custom Function

The **Invoke Custom Function** feature allows users to apply a pre-defined custom M function to each row of a table.



Understanding Custom Functions

Before invoking a function, you must **create** it. Functions in Power Query are written in **M language**, also called Mashup language.

Where Are Custom Functions Useful?

- Forecasting or regression models
- Time-series transformations
- Custom domain-specific calculations
- API-driven lookups
- Complex reusable logic across datasets

Steps to Use Invoke Custom Function:

1. Create a **blank query**.
2. Write M language code that defines a function, e.g.:

```
(x) => x * 1.2
```

YATICORP
EMPOWERING EDUCATION

3. Name the query (e.g., *SalesMultiplier*). Power Query recognizes it as a function.
4. Return to your main dataset.
5. Choose **Invoke Custom Function**.
6. Select the function and pass parameters (columns or values).

Example Scenario:

If the sales team had built a predictive model externally and converted it to an M function, the model could be plugged into Power Query and applied to each row of sales data.

---

7. Column From Examples

Column From Examples is a very powerful semi-automated transformation tool. It allows users to create new derived columns by simply typing sample output values. Power Query learns the pattern and generates the M code automatically.

What This Feature Can Do:

- Extract substrings
- Combine multiple fields
- Clean text
- Convert formats
- Apply mathematical transformations
- Detect and reproduce patterns

YATICORP
EMPOWERING EDUCATION

Usage Variants:

You can create:

- **A new column from selected columns**
- **A new column from entire table**

Example – Mathematical Transformations

Selecting the *Quantity* column and choosing **Column From Examples → From Selection**, Power Query offers suggestions such as:

- Square
- Square root
- Cube
- Logarithm
- Trigonometric functions

- Length (number of characters)

After entering sample values, Power Query identifies the pattern and fills the entire column.

Benefits:

- No coding required
- Great for quick prototyping
- Works with both text and numeric data
- Generates clean, editable M code

Example – Multiple Column Selection

When selecting 2–3 columns and using *Column From Examples*, the suggestions expand to include:

- Concatenations
- Date/time extractions
- Sign (positive/negative)
- Custom mathematical operations

YATICORP
EMPOWERING EDUCATION

---

8. Manage Parameters

This is one of the most advanced and practical features in Power Query. Parameters allow users to store single values that can be referenced across multiple queries.

Why Use Parameters?

- To create reusable variables
- To make queries dynamic
- To perform threshold-based analysis
- To avoid hard-coding values

- To centrally manage frequently used constants

Example Scenario – Minimum Sales Quantity Benchmark

Assume the company has set a benchmark:

Minimum quantity per order = **3**

Instead of typing “3” many times, a parameter is created.

Steps:

1. Go to **Home → Manage Parameters**.
2. Click **New Parameter**.
3. Configure fields:
    - **Name:** Minimum_Sales_Quantity
    - **Type:** Decimal Number / Whole Number
    - **Current Value:** 3

YATICORP
EMPOWERING EDUCATION

The parameter now appears in the Queries pane.

Using the Parameter in Conditional Logic:

A new column can be created using **Add Column → Conditional Column**:

- If Quantity ≥ Minimum_Sales_Quantity → "Yes – Above Benchmark"
- Else → "No – Below Benchmark"

This automatically updates if the parameter's value changes.

Advantages of Parameterization:

- Queries become versatile and reusable.
- Changing one value updates the entire model.
- Excellent for business rule-driven datasets.
- Useful for controlling filters, thresholds, ranges, and API endpoints.

9. Practical Examples Where Parameters Are Useful

1. Dynamic Date Range Filtering

You can create parameters:

- Start_Date
- End_Date

Then filter rows between these dates.

2. Dynamic Currency Conversion

Parameter: Exchange_Rate
Used in multiplying cost/sales values automatically.

3. Switching Data Sources

You can parameterize:

- File paths
- Folder paths
- API URLs

YATICORP
EMPOWERING EDUCATION

This makes the report maintainable.

4. Sensitivity Analysis

E.g., reorder point for supply chain, safety stock percentage, discount rates—all can be parameters.

---

10. Completing the Transformations

Once all transformations are complete:

- Click **Close & Apply** in Power Query.
- Data loads into Power BI Desktop.
- The transformed columns and parameters are now available for:
    - Data modeling
    - DAX calculations
    - Visualizations
    - Dashboard creation

YATICORP
EMPOWERING EDUCATION